

UNIVERSIDADE DE SÃO PAULO (USP)
INSTITUTO DE CIÊNCIAS MATEMÁTICAS
E DE COMPUTAÇÃO (ICMC)
SÃO CARLOS, SP

Autores (N°USP):
Henrique Mohr (18130304)
Pedro Henrique Costa Teixeira (18131584)
Henrique Rossi Posso (17868285)

Relatório do Projeto da Mini CPU

Objetivo: O objetivo principal do projeto é projetar, implementar e validar o funcionamento de uma Mini CPU de 16 bits

Disciplina: Prática em Sistemas Digitais (SSC0108)
Docente: Denis Fernando Wolf

13 de outubro de 2026



SUMÁRIO

1. INTRODUÇÃO **3**
1.0 Objetivo do Projeto 3
2. DESENVOLVIMENTO **3**
2.0 Processamento Aritmético e Lógico: 3
2.1 Gerenciamento de Registradores Internos: 3
2.2 Controle de Fluxo e Tomada de Decisão: 3
2.3 Tratamento de Sinalização e Reinicialização: 4
2.4 Mapeamento Geral da Palavra de Instrução (Bits 15 a 0) 4
3. CONCLUSÃO **5**
4. REFERÊNCIAS **6**

1. INTRODUÇÃO

1.0 Objetivo do Projeto

O objetivo principal do projeto é projetar, implementar e validar o funcionamento de uma **Mini CPU de 16 bits** (WIDTH=16) com suporte a uma memória de programa de **64 posições** (DEPTH=64), capaz de executar sequencialmente um conjunto de instruções para processamento aritmético, lógico e controle de fluxo.

2. DESENVOLVIMENTO

2.0 Processamento Aritmético e Lógico:

- Execução de operações aritméticas básicas de adição, subtração, multiplicação e **divisão** entre registradores internos e valores imediatos/constantes.
- Execução de operações lógicas bit a bit (**AND**, **OR**, **XOR**) e determinação de valor máximo (**MAX**).

2.1 Gerenciamento de Registradores Internos:

- Leitura, escrita e atualização correta do banco de registradores gerais da CPU (registradores **A**, **B** e **C**).
- Capacidade de zeramento individual de registradores via instrução aritmética (ex: A = A - A).

2.2 Controle de Fluxo e Tomada de Decisão:

- Implementação e execução correta de desvios condicionais e incondicionais (JUMP IF EQUAL, JUMP IF ZERO e JUMP INCONDICIONAL).
- Utilização do módulo de comparação (COMPARA) para avaliação de condições relativas entre operandos (identificação de igualdade ou magnitude para acionamento de saídas/LEDs).
- Construção e validação de rotinas estruturadas de repetição (loops).

2.3 Tratamento de Sinalização e Reinicialização:

- Validação de robuster no decodificador de instruções para evitar falsos acionamentos por sobreposição de padrões de bits (ex: diferenciação entre um endereço de JUMP e o código de *Master Reset*).
- Implementação da instrução de **Master Reset**, capaz de redefinir o estado dos registradores gerais (A=0, B=0, C=0) e reiniciar o ciclo de busca e execução a partir do endereço de memória 0

2.4 Mapeamento Geral da Palavra de Instrução (Bits 15 a 0)

B15	B14	B13	B12	B11	B10	B9	B8	B7	B6	B5	B4	B3	B2	B1	B0

- B15 – B14: selecionam a operação aritmética entre SOMA, SUBTRAÇÃO, DIVISÃO e MULTIPLICAÇÃO
  i. (00) = SOMA
  ii. (01) = SUBTRAÇÃO
  iii. (10) = MULTIPLICAÇÃO
  iv. (11) = DIVISÃO (A operação de divisão sempre pegará a origem 2 como dividendo e a origem 1 como divisor)
- B13 – B12: seleção do registrador um (origem), podendo ser o registrador A, B ou o C.
  i. (00) = Registrador A
  ii. (01) = Registrador B
  iii. (10) = Registrador C
  iv. (11) = Constante
- B11 – B10: seleção do registrador dois (origem), podendo ser, também, o registrador A, B ou C
  i. (00) = Registrador A
  ii. (01) = Registrador B
  iii. (10) = Registrador C
  iv. (11) = Constante
- B9 – B8: seleção do registrador de destino, o registrador três, podendo ser o registrador A, B ou C
  i. (00) = Registrador A
  ii. (01) = Registrador B
  iii. (10) = Registrador C
  iv. (11) = Inválido (usado quando não precisa mudar nada nos registradores).

- B7 – B6: seleciona a operação de jump, podendo ser JUMP INCONDICIONAL, JUMP IF ZERO ou JUMP IF EQUAL.
    i. (00) = Inválido
    ii. (01) = JUMP INCONDICIONAL
    iii. (10) = JUMP IF ZERO
    iv. (11) = JUMP IF EQUAL

    OBS: Os jumps, tem preferência sobre as outras operações, mesmo se tiver uma "master reset", o jump será executado, além das outras operações. Preferência máxima

- B5 – B4: selecionam as operações de MASTER RESET, COMPARADOR e a OPERAÇÃO NOVA
    i. (00) = Inválido
    ii. (01) = Master Reset
    iii. (10) = Comparar
    iv. (11) = Operação Nova (ativa a calculadora lógica e utiliza os bits 15-14 para determinar qual das operações lógicas será feita)
        1. (00) = AND
        2. (01) = OR
        3. (10) = XOR
        4. (11) = MAX (pega o maior dos números dos registradores e joga no registrador destino)
- B3 – B0: constante de 4 bits (0 a 15), usada quando uma origem é `11`. O mesmo valor é enviado às duas origens.
- OPERAÇÃO MOV: não há instrução própria. A movimentação usa a SOMA com zero:

    Registrador → registrador: origem 1 = registrador de origem, origem 2 = constante `0000`. Exemplo: `MOV B, A` = `0000 1101 0000 0000` (0x0D00).

    Constante → registrador: como as duas origens compartilham B3–B0, a origem 1 recebe a constante e a origem 2 recebe um registrador que vale zero (obtido com `X = X - X` ou com o Master Reset). Exemplo, com B = 0: `MOV A, #5` = `0011 0100 0000 0101`.

3. CONCLUSÃO

O desenvolvimento e a validação do projeto da Mini CPU de 16 bits comprovam a eficiência e a precisão da arquitetura proposta para o processamento de dados e o controle de rotinas. O sistema demonstrou capacidade plena para executar operações aritméticas fundamentais — com destaque para o alinhamento e mapeamento correto dos operandos de divisão —, além de operações lógicas avançadas (AND, OR, XOR e MAX).

A estrutura de controlo de fluxo mostrou-se robusta e fiável na tomada de decisões condicionais (JUMP IF EQUAL e JUMP IF ZERO), no redirecionamento incondicional de rotinas e na gestão de laços de repetição (loops). A validação do descodificador assegurou também a imunidade do circuito a conflitos entre endereços de desvio e instruções especiais. Por fim, a execução bem-sucedida do Master Reset garante a capacidade do processador em restabelecer o seu estado inicial (A=0, B=0, C=0) e reiniciar o ciclo de execução de forma estável, consolidando o projeto como uma solução funcional e coerente.

4. REFERÊNCIAS

BROWN, S.; VRANESIC, Z. Fundamentals of Digital Logic with VHDL Design, 2nd Edition, McGraw Hill, 2004, 939pg.

BROWN, S.; VRANESIC, Z. Fundamentals of Digital Logic with VHDL Design, 3th Edition, McGraw Hill, 2014, 864pg.

IDOETA, I.V.; CAPUANO, F.G. Elementos de Eletrônica Digital, Livros Érica, 2007